

Analisis

Link al repositorio con los desafios:

<https://github.com/Facundo-Mourelle/lab2-RTOS>

Desafio 1

Si la tarea consumidora es más lenta que la productora, la cola comenzará a llenarse gradualmente con cada pulsación del botón. Una vez que la cola alcance su capacidad máxima, el comportamiento del sistema dependerá del parámetro `xTicksToWait` de `xQueueSend`. Si la cola está llena, el productor se bloqueará intentando insertar el dato, y si el tiempo expira y el consumidor aún no ha liberado espacio, `xQueueSend` fallará, provocando la pérdida de ese evento de pulsación. Sin embargo, el contador interno del productor avanzará, pero los LEDs eventualmente recibirán el valor actual y mostrarán ese número correcto a pesar de haberse saltado los valores anteriores.

El tamaño de la cola determina directamente la capacidad del sistema para procesar los valores en caso de que el productor sea notablemente más rápido que el consumidor. Esto es porque permite que el productor registre múltiples clics rápidos consecutivos sin perder eventos, dándole tiempo al consumidor lento para progresivamente procesar los datos almacenados. Por el contrario, una cola demasiado pequeña satura rápidamente el buffer, incrementando la probabilidad de pérdida de datos.

```

#define LED1 GPIO_PIN_12
#define LED2 GPIO_PIN_13
#define LED3 GPIO_PIN_14
#define LED4 GPIO_PIN_15
#define LED_PORT GPIOD

#define BTN_PIN GPIO_PIN_0
#define BTN_PORT GPIOA

static QueueHandle_t xQueuePulsador;

void vSourceTask(void *pvParameters) {
    uint16_t contador = 0;
    GPIO_PinState estadoAnterior = GPIO_PIN_RESET;
    for (;;) {
        GPIO_PinState estadoActual = HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN);
        if (estadoActual == GPIO_PIN_SET && estadoAnterior == GPIO_PIN_RESET) {
            vTaskDelay(pdMS_TO_TICKS(50));
            /* si se mantiene el estado => boton efectivamente presionado */
            if (HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN) == GPIO_PIN_SET)
                contador++;
            xQueueSend(xQueuePulsador, &contador, pdMS_TO_TICKS(10));
            while (HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN) == GPIO_PIN_SET) {
                vTaskDelay(pdMS_TO_TICKS(10));
            }
        }
        estadoAnterior = estadoActual;
        vTaskDelay(pdMS_TO_TICKS(20));
    }
}

void vSetLeds(uint16_t led_value) {
    HAL_GPIO_WritePin(LED_PORT, LED1,
        (led_value & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, LED2,
        (led_value & 0x02) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, LED3,
        (led_value & 0x04) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, LED4,
        (led_value & 0x08) ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

void vSinkTask(void *pvParameters) {
    uint16_t contador_recibido;
    for (;;) {
        /* bloquea hasta recibir nuevos datos por queue */
        if (xQueueReceive(xQueuePulsador, &contador_recibido, portMAX_DELAY) ==
            pdTRUE) {
            vSetLeds(contador_recibido);
        }
    }
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    xQueuePulsador = xQueueCreate(10, sizeof(uint16_t));
    if (xQueuePulsador != NULL) {
        xTaskCreate(vSourceTask, "Source", 128, NULL, 1, NULL);
        xTaskCreate(vSinkTask, "Sink", 128, NULL, 1, NULL);
    }
    MX_FREERTOS_Init();
    osKernelStart();
    while (1) {
    }
}

```

Desafío 2

Enviar estructuras por copia implica que FreeRTOS reserva memoria en el heap para almacenar el valor de cada mensaje ingresado a la queue. Para estructuras pequeñas el impacto en memoria es mínimo, y tiene como ventaja la seguridad, ya que los datos quedan aislados. La tarea de periféricos puede modificar su variable local `msg` inmediatamente después del `xQueueSend` sin afectar el dato que la tarea principal procesara.

Enviar estructuras por puntero reduce el tamaño del elemento en la cola a una dirección de memoria (típicamente 4 bytes), lo cual es crítico al transmitir cargas grandes. Sin embargo, introduce severos desafíos de gestión de memoria, lo que añade complejidad innecesaria para mensajes si se manejan elementos pequeños.

```

typedef enum { APAGAR = 0, ENCENDER = 1 } led_cmd_t;

typedef struct {
    uint8_t led_id;
    led_cmd_t command;
} led_message_t;

#define LED1 GPIO_PIN_12
#define LED2 GPIO_PIN_13
#define LED3 GPIO_PIN_14
#define LED4 GPIO_PIN_15
#define LED_PORT GPIOC

QueueHandle_t xQueueLeds;
uint16_t pines[4] = {LED1, LED2, LED3, LED4};

void vMainTask(void *pvParameters) {
    led_message_t msg;

    for (;;) {
        // Efecto cascada
        for (int8_t i = 0; i < 4; i++) {
            msg.led_id = i;
            msg.command = ENCENDER;
            xQueueSend(xQueueLeds, &msg, portMAX_DELAY);
            vTaskDelay(pdMS_TO_TICKS(100));

            msg.command = APAGAR;
            xQueueSend(xQueueLeds, &msg, portMAX_DELAY);
        }
        vTaskDelay(pdMS_TO_TICKS(500));

        // Efecto parpadeo 3 veces
        for (int parpadeo = 0; parpadeo < 3; parpadeo++) {
            // Encender
            for (uint8_t i = 0; i < 4; i++) {
                msg.led_id = i;
                msg.command = ENCENDER;
                xQueueSend(xQueueLeds, &msg, portMAX_DELAY);
            }
            vTaskDelay(pdMS_TO_TICKS(200));

            // Apagar
            for (uint8_t i = 0; i < 4; i++) {
                msg.led_id = i;
                msg.command = APAGAR;
                xQueueSend(xQueueLeds, &msg, portMAX_DELAY);
            }
            vTaskDelay(pdMS_TO_TICKS(200));
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void vSideTask(void *pvParameters) {
    led_message_t msg;

    for (;;) {
        // bloquear hasta recibir un mensaje
        if (xQueueReceive(xQueueLeds, &msg, portMAX_DELAY) == pdTRUE) {
            if (msg.led_id < 4) {
                GPIO_PinState estado =
                    (msg.command == ENCENDER) ? GPIO_PIN_SET : GPIO_PIN_RESET;
                HAL_GPIO_WritePin(LED_PORT, pines[msg.led_id], estado);
            }
        }
    }
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    xQueueLeds = xQueueCreate(20, sizeof(led_message_t));
    if (xQueueLeds != NULL) {
        xTaskCreate(vMainTask, "MainTask", 128, NULL, 1, NULL);
        xTaskCreate(vSideTask, "PeriTask", 128, NULL, 2, NULL);
    }
    MX_FREERTOS_Init();
    osKernelStart();
    while (1) {
    }
}

```

Desafío 3

La ventaja principal de un QueueSet frente al polling con xQueueReceive es la eficiencia en la gestión del Scheduler. En un esquema de polling, la tarea procesadora tendría que alternar entre estados Ready y Running continuamente solo para revisar si hay datos, o requeriría un vTaskDelay explícito que introduciría latencia. Con un QueueSet, la tarea entra en estado Blocked hasta que ingrese un nuevo dato a la queue o expire el temporizador. Esto garantiza una respuesta determinista inmediata a los eventos, liberando ciclos de reloj.

El tamaño de las queues individuales impacta severamente la estructura del Queue Set debido a su implementación. Como la capacidad del QueueSet debe ser igual o mayor que la suma de las longitudes de todas las colas y semáforos que lo componen, dedicar mayor tamaño a las queues individuales se fuerza al sistema a reservar un bloque contiguo de memoria en el heap, reduciendo drásticamente la eficiencia.

```

#define QUEUE_VELOCIDAD_LEN 3
#define QUEUE_MODALEN 3

#define LED1_GPIO_PIN_12
#define LED2_GPIO_PIN_13
#define LED3_GPIO_PIN_14
#define LED4_GPIO_PIN_15
#define LED_PORT_GPIO0

#define BTN_PIN_GPIO_PIN_0
#define BTN_PORT_GPIOA

static QueueHandle_t Cola_Velocidad;
static QueueHandle_t Cola_Modo;
static QueueSetHandle_t xQueueSet;

void vSetLeds(uint16_t value) {
    HAL_GPIO_WritePin(LED_PORT, LED1,
        (value & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, LED2,
        (value & 0x02) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, LED3,
        (value & 0x04) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, LED4,
        (value & 0x08) ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

void vVelocidadTask(void *pvParameters) {
    uint16_t velocidad;
    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(5000));
        // Generar valor entre 100 y 300 ms
        velocidad = 100 + (rand() % 201);
        xQueueSend(Cola_Velocidad, &velocidad, 0);
    }
}

void vModoTask(void *pvParameters) {
    GPIO_PinState estadoAnterior = GPIO_PIN_RESET;
    char modo_actual = 'D';

    for (;;) {
        GPIO_PinState estadoActual = HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN);
        if (estadoActual == GPIO_PIN_SET && estadoAnterior == GPIO_PIN_RESET) {
            vTaskDelay(pdMS_TO_TICKS(50));
            estadoActual = HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN);

            if (estadoActual == GPIO_PIN_SET) {
                modo_actual = (modo_actual == 'D') ? 'I' : 'D';
                xQueueSend(Cola_Modo, &modo_actual, pdMS_TO_TICKS(10));
            }
            estadoAnterior = estadoActual;
            vTaskDelay(pdMS_TO_TICKS(20));
        }
    }
}

void vProcessTask(void *pvParameters) {
    uint16_t current_speed = 200;
    char current_mode = 'D';
    uint8_t led_state = 0x01; // Secuencia inicial (LED0 encendido)
    QueueSetMemberHandle_t xActivatedMember;

    for (;;) {
        // Bloquear en el Queue Set. El timeout se usa como delay
        xActivatedMember =
            xQueueSelectFromSet(xQueueSet, pdMS_TO_TICKS(current_speed));

        if (xActivatedMember == Cola_Velocidad) {
            xQueueReceive(Cola_Velocidad, &current_speed, 0);
        } else if (xActivatedMember == Cola_Modo) {
            xQueueReceive(Cola_Modo, &current_mode, 0);
        } else if (xActivatedMember == NULL) {
            // El timeout expiró => avanzar el frame de la secuencia LED
            vSetLeds(led_state);
            if (current_mode == 'D') {
                led_state = (led_state >> 1) | ((led_state & 0x01) << 3);
            } else {
                led_state = (led_state << 1) | ((led_state & 0x08) >> 3);
            }
        }
    }
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    Cola_Velocidad = xQueueCreate(QUEUE_VELOCIDAD_LEN, sizeof(uint16_t));
    Cola_Modo = xQueueCreate(QUEUE_MODALEN, sizeof(char));
    xQueueSet = xQueueCreateSet(QUEUE_VELOCIDAD_LEN + QUEUE_MODALEN);
    if (xQueueSet != NULL && Cola_Velocidad != NULL && Cola_Modo != NULL) {
        xQueueAddToSet(Cola_Velocidad, xQueueSet);
        xQueueAddToSet(Cola_Modo, xQueueSet);
        xTaskCreate(vVelocidadTask, "Velocidad", 128, NULL, 1, NULL);
        xTaskCreate(vModoTask, "Modo", 128, NULL, 1, NULL);
        xTaskCreate(vProcessTask, "Procesadora", 128, NULL, 2, NULL);
    }
    MX_FREERTOS_Init();
    osKernelStart();
    while (1) {
    }
}

```

Desafío 4

Si una tarea usa `xQueueReceive()` en lugar de `xQueuePeek()`, la cola queda vacía inmediatamente después de que la primera tarea (digamos A) consume el dato. La Tarea B entra en `xQueuePeek()` y queda bloqueada indefinidamente porque no hay nada que leer. El dato se pierde para B. Para asegurar que ambas tareas leyeron el mismo dato antes de eliminarlo se emplean los dos semáforos binarios. Ninguna tarea puede “confirmar antes de leer” porque el `xSemaphoreGive()` se ejecuta después del `xQueuePeek()` y del procesamiento del LED. De esta forma la tarea de limpieza no puede avanzar hasta que ambos semáforos estén disponibles.

```

#define LED1 GPIO_PIN_12
#define LED2 GPIO_PIN_13
#define LED_PORT GPIOD

#define BTN_PIN GPIO_PIN_0
#define BTN_PORT GPIOA

static QueueHandle_t Cola_Global = NULL;
static SemaphoreHandle_t Sem_LeidoA = NULL;
static SemaphoreHandle_t Sem_LeidoB = NULL;

void vTareaProductora(void *pvParameters) {
    uint16_t contador = 0;
    uint8_t estado_ant = GPIO_PIN_RESET;
    for (;;) {
        uint8_t estado_act = HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN);
        if (estado_act == GPIO_PIN_RESET && estado_ant == GPIO_PIN_SET) {
            vTaskDelay(pdMS_TO_TICKS(50));
            contador++;
            xQueueOverwrite(Cola_Global, &contador);
        }
        estado_ant = estado_act;
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

void vTareaA(void *pvParameters) {
    uint16_t valor;
    for (;;) {
        // Blocked hasta haber datos, pero no los extrae
        if (xQueuePeek(Cola_Global, &valor, portMAX_DELAY) == pdTRUE) {
            if (valor % 2 == 0) {
                for (int i = 0; i < 3; i++) {
                    HAL_GPIO_WritePin(LED_PORT, LED1, GPIO_PIN_SET);
                    vTaskDelay(pdMS_TO_TICKS(200));
                    HAL_GPIO_WritePin(LED_PORT, LED1, GPIO_PIN_RESET);
                    vTaskDelay(pdMS_TO_TICKS(200));
                }
                xSemaphoreGive(Sem_LeidoA);
                taskYIELD();
            }
        }
    }
}

void vTareaB(void *pvParameters) {
    uint16_t valor;
    for (;;) {
        // Blocked hasta haber datos, pero no los extrae
        if (xQueuePeek(Cola_Global, &valor, portMAX_DELAY) == pdTRUE) {
            if (valor % 2 != 0) {
                for (int i = 0; i < 3; i++) {
                    HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_SET);
                    vTaskDelay(pdMS_TO_TICKS(200));
                    HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_RESET);
                    vTaskDelay(pdMS_TO_TICKS(200));
                }
                xSemaphoreGive(Sem_LeidoB);
                taskYIELD();
            }
        }
    }
}

void vTareaLimpieza(void *pvParameters) {
    uint16_t valor_descartado;
    for (;;) {
        xSemaphoreTake(Sem_LeidoA, portMAX_DELAY);
        xSemaphoreTake(Sem_LeidoB, portMAX_DELAY);
        // queue deberia siempre tener un dato al llegar aca
        xQueueReceive(Cola_Global, &valor_descartado, 0);
        // queue ahora vacia
    }
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    Cola_Global = xQueueCreate(1, sizeof(uint16_t));
    if (Cola_Global != NULL) {
        /* Semáforos binarios (estado inicial = no disponible) */
        Sem_LeidoA = xSemaphoreCreateBinary();
        Sem_LeidoB = xSemaphoreCreateBinary();
        configASSERT(Sem_LeidoA != NULL && Sem_LeidoB != NULL);

        xTaskCreate(vTareaProductora, "Productora", 128, NULL, 3, NULL);
        xTaskCreate(vTareaA, "TareaA", 128, NULL, 2, NULL);
        xTaskCreate(vTareaB, "TareaB", 128, NULL, 2, NULL);
        xTaskCreate(vTareaLimpieza, "Limpieza", 128, NULL, 1, NULL);
    }
    MX_FREERTOS_Init();
    osKernelStart();
    while (1) {
    }
}

```


Desafío 5

Si el usuario presiona el botón repetidamente cada 2 segundos, el LED2 permanecerá encendido indefinidamente y el LED3 nunca se activará. Esto ocurre porque la llamada a `xTimerResetFromISR` reinicia el contador de ticks del timer a cero antes de que alcance los 5 segundos.

El LED1 no se ve afectado por la lógica del LED2 ni de los temporizadores, ya que está encapsulado en su propia tarea. Dado que el sistema está configurado con un planificador preemptivo, la gestión de la queue del temporizador y la ejecución de la ISR toman la CPU por momentos cortos, pero no bloquean ni alteran el flujo de la tarea del LED1.

La función de callback del temporizador no debe usar `vTaskDelay()` ni ninguna función bloqueante. Los callbacks de todos los Software Timers se ejecutan en el contexto de una única tarea compartida del sistema. Si se bloquea esta tarea se bloquea todo el subsistema de temporizadores, por lo que ningún otro temporizador podrá expirar y ningún comando de temporizador enviado por otras tareas será procesado. Por eso se utilizó un segundo temporizador para mantener el LED3 encendido.

```

#define LED1 GPIO_PIN_12
#define LED2 GPIO_PIN_13
#define LED3 GPIO_PIN_14
#define LED_PORT GPIOD

#define BTN_PIN GPIO_PIN_0
#define BTN_PORT GPIOA

static TimerHandle_t xWatchdogTimer;
static TimerHandle_t xLed30ffTimer;

void vLed1Task(void *pvParameters) {
    for (;;) {
        HAL_GPIO_TogglePin(LED_PORT, LED1);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void vLed3Callback(TimerHandle_t xTimer) {
    HAL_GPIO_WritePin(LED_PORT, LED3, GPIO_PIN_RESET);
}

void vWatchdogCallback(TimerHandle_t xTimer) {
    HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_RESET);

    // usar vTaskDelay bloquearia la tarea
    // => usamos otro timer para apagar el LED3
    HAL_GPIO_WritePin(LED_PORT, LED3, GPIO_PIN_SET);

    xTimerStart(xLed30ffTimer, 0);
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if (GPIO_Pin == BTN_PIN) {
        HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_SET);

        xTimerResetFromISR(xWatchdogTimer, &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    xWatchdogTimer = xTimerCreate("WDG_Timer", pdMS_TO_TICKS(5000), pdFALSE,
        (void *)0, vWatchdogCallback);

    xLed30ffTimer = xTimerCreate("LED3_Timer", pdMS_TO_TICKS(1000), pdFALSE,
        (void *)1, vLed3Callback);

    if (xWatchdogTimer != NULL && xLed30ffTimer != NULL) {
        xTaskCreate(vLed1Task, "LED1_Task", 128, NULL, 1, NULL);
    }
    MX_FREERTOS_Init();
    osKernelStart();
    while (1) {
    }
}

```

Desafío 6

Un Software Timer es mucho más eficiente en memoria que una tarea dedicada que realiza en bucle utilizando `vTaskDelay` porque no posee un stack propio, ya que utiliza el de la Timer Daemon Task, lo que ahorra memoria significativamente.

La lógica de conmutación del LED reside en la función de callback del temporizador, por lo que al no ser una tarea el Timer Daemon Task invoca al callback cada vez que el auto-reload expira.

```

#define LED4 GPIO_PIN_15
#define LED_PORT GPIOD

#define BTN_PIN GPIO_PIN_0
#define BTN_PORT GPIOA

#define PERIOD_MAX_MS 1000U
#define PERIOD_MIN_MS 125U

static TimerHandle_t xMetronomeTimer = NULL;
static volatile uint32_t ulCurrentPeriod = PERIOD_MAX_MS;

static void vMetronomeCallback(TimerHandle_t xTimer) {
    HAL_GPIO_TogglePin(LED_PORT, LED4);
}

static void vButtonTask(void *pvParameters) {
    GPIO_PinState xLastState = GPIO_PIN_RESET;
    GPIO_PinState xCurrentState;
    for (;;) {
        xCurrentState = HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN);

        if (xCurrentState == GPIO_PIN_SET && xLastState == GPIO_PIN_RESET) {
            // dividir frecuencia a la mitad
            ulCurrentPeriod /= 2U;
            if (ulCurrentPeriod < PERIOD_MIN_MS) {
                ulCurrentPeriod = PERIOD_MAX_MS;
            }

            xTimerChangePeriod(xMetronomeTimer, pdMS_TO_TICKS(ulCurrentPeriod),
                               pdMS_TO_TICKS(10));
        }

        xLastState = xCurrentState;
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    xMetronomeTimer = xTimerCreate("Metronome", pdMS_TO_TICKS(PERIOD_MAX_MS),
                                   pdTRUE, (void *)0, vMetronomeCallback);

    if (xMetronomeTimer != NULL) {
        xTimerStart(xMetronomeTimer, 0);
        xTaskCreate(vButtonTask, "Button", configMINIMAL_STACK_SIZE, NULL, 2, NULL);
    }
    MX_FREERTOS_Init();
    osKernelStart();
    while (1) {
    }
}

```

Desafío 7

Los temporizadores no se comunican directamente entre si, ya que los callbacks lo hacen a través del contexto del Timer Daemon Task. La comunicación con el resto del sistema se realiza mediante la variable *ucSystemArming* y las versiones FromISR de la API de timers, que internamente envían comandos a la cola interna del daemon task en lugar de ejecutarlos de forma inmediata.

Es seguro llamar a `xTimerStop()`, `xTimerStart()` o `xTimerChangePeriod()` desde el callback de otro timer, porque estas funciones no operan directamente sobre las estructuras del timer sino que encolan un comando en la timer command queue. Luego, el daemon task los procesará en orden una vez que el callback actual retorne. Lo que no es seguro es usar las variantes FromISR dentro de un callback, ya que esas funciones están diseñadas exclusivamente para contextos de interrupción, no para el contexto de tarea en que corren los callbacks.

```

#define LED1 GPIO_PIN_12
#define LED2 GPIO_PIN_13
#define LED_PORT GPIOD

#define BTN_PIN GPIO_PIN_0
#define BTN_PORT GPIOA

#define BLINK_PERIOD_MS pdMS_TO_TICKS(200)
#define BUILD_TIMEOUT_MS pdMS_TO_TICKS(10000)

static TimerHandle_t xBlinkTimer;
static TimerHandle_t xBuildTimer;
/* 0: idle , 1: armando alarma */
static volatile uint8_t ucSystemBuilding = 0;

static void vBlinkCallback(TimerHandle_t xTimer) {
    HAL_GPIO_TogglePin(LED_PORT, LED1);
}

static void vBuildCallback(TimerHandle_t xTimer) {
    /*no estamos en interrupcion => llamada comun*/
    xTimerStop(xBlinkTimer, 0);
    HAL_GPIO_WritePin(LED_PORT, LED1, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_SET);
    ucSystemBuilding = 0;
}

void vButtonISR(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if (ucSystemBuilding) {
        /*cancelar armado */
        /*FromISR porque estamos en una interrupcion */
        xTimerStopFromISR(xBlinkTimer, &xHigherPriorityTaskWoken);
        xTimerStopFromISR(xBuildTimer, &xHigherPriorityTaskWoken);
        HAL_GPIO_WritePin(LED_PORT, LED1, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_RESET);
        ucSystemBuilding = 0;
    } else {
        /*empezar armado */
        ucSystemBuilding = 1;
        /*LED2 pudo haber estado encendido si el sistema estaba armado */
        HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_RESET);
        xTimerStartFromISR(xBlinkTimer, &xHigherPriorityTaskWoken);
        xTimerStartFromISR(xBuildTimer, &xHigherPriorityTaskWoken);
    }

    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    xBlinkTimer =
        xTimerCreate("Blink", BLINK_PERIOD_MS, pdTRUE, NULL, vBlinkCallback);
    xBuildTimer =
        xTimerCreate("Build", BUILD_TIMEOUT_MS, pdFALSE, NULL, vBuildCallback);

    if (xBlinkTimer != NULL && xBuildTimer != NULL) {
        /* asegurar el estado inicial */
        ucSystemBuilding = 0;
        HAL_GPIO_WritePin(LED_PORT, LED1, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED_PORT, LED2, GPIO_PIN_RESET);
    }
    MX_FREERTOS_Init();
    osKernelStart();
    while (1) {
    }
}

```